



UNIVERSIDAD DE EL SALVADOR
FACULTAD DE INGENIERÍA Y ARQUITECTURA
ESCUELA DE INGENIERÍA DE SISTEMAS INFORMÁTICOS

**Clasificación de Enfermedades, Plagas y Deficiencias
Nutricionales en Hojas de Maíz mediante Modelos
Convolutionales Ligeros: Recolección de Datos, EDA y Baselines**

Etapas 1

Integrantes:

Josias Abner Rivas Fuentes – RF20010
David Alejandro Deras Cerros – DC19019
Elmer Edenilson Rosales Molina – RM20001

San Salvador, El Salvador
July 11, 2026

Índice

Resumen	2
1 Problema y Solución	3
2 Trabajos Previos	3
3 Marco Teórico	4
3.1 Redes Neuronales Convolucionales	4
3.2 Evolución hacia arquitecturas ligeras	5
3.3 Aprendizaje por Transferencia	6
3.4 Baselines	6
3.5 Desbalance de clases	6
3.6 Métricas de evaluación	7
3.7 Interpretabilidad	7
4 Descripción del Dataset	9
5 Análisis Exploratorio de Datos	10
6 Preprocesamiento, Feature Engineering y Estrategia	13
6.1 Normalización y división	13
6.2 Balanceo	14
6.3 Baselines elegidos y por qué	14
6.4 Estrategia de entrenamiento: baselines vs. principal	15
7 Pipeline de Baselines y Resultados	16
7.1 Dataset y configuración	16
7.2 Resultados globales	17
7.3 Dónde falla el modelo	17
7.4 Interpretabilidad aplicada	17
7.5 Reproducibilidad	19
8 Conclusiones y Trabajo Futuro	19
9 Referencias	20

Resumen

Este trabajo presenta la Etapa 1 de un sistema de clasificación de enfermedades foliares, plagas y deficiencias nutricionales en maíz mediante *deep learning*, pensado para inferencia offline en dispositivos móviles de gama media/baja. Se documenta la recolección y limpieza de un conjunto consolidado de 31 622 imágenes en 9 clases procedentes de 6 fuentes públicas, un análisis exploratorio que revela un desbalance de hasta $32.9\times$ y sesgos de dominio laboratorio/campo, el pipeline de preprocesamiento y balanceo, y la comparación de tres arquitecturas convolucionales ligeras preentrenadas en ImageNet (EfficientNet-B0, ShuffleNetV2-x1.0 y EfficientNet-Lite0) sobre las 9 clases con un tope de 1 500 imágenes por clase. EfficientNet-B0 obtiene el mejor macro-F1 en prueba (0.9146), seguido de ShuffleNetV2-x1.0 (0.9030) y EfficientNet-Lite0 (0.8951); los tres superan la meta de macro-F1 ≥ 0.85 . El error se concentra en un solo cuello de botella, `potassium_deficiency`, y en la confusión entre lesiones foliares parecidas. El análisis de interpretabilidad con LIME y Grad-CAM muestra que los modelos atienden al tejido foliar, pero se equivocan con alta confianza dentro del bloque de deficiencias.

Palabras clave: clasificación de enfermedades foliares, maíz, redes neuronales convolucionales, transfer learning, interpretabilidad, dispositivos móviles

Enlaces del proyecto. El código, los datos y la documentación ampliada de este trabajo están disponibles públicamente:

- Repositorio: <https://github.com/daiv05/corn-leaf-disease-project>
- Sitio del proyecto: <https://doctor-maiz.deras.dev/es>

1 Problema y Solución

La agricultura representa el 5.6% del PIB de El Salvador y sostiene a más de 2 millones de personas en zonas rurales; el 82.1% de los productores son pequeños agricultores, muchos de subsistencia. El maíz, cultivo principal, es vulnerable a enfermedades foliares, plagas y deficiencias nutricionales que, sin detección temprana, pueden destruir hasta el 70% de la cosecha. Entre 2021 y 2023 la cosecha nacional cayó un tercio.

El problema de fondo es de acceso. En zonas rurales la asistencia técnica agrícola escasea, así que el diagnóstico depende de la experiencia de quien cultiva, lo que favorece detecciones tardías. Este proyecto no busca reemplazar al extensionista, sino ofrecer una **herramienta de apoyo** que detecte a tiempo una condición foliar y oriente sobre próximos pasos.

La solución es un clasificador de imágenes liviano, capaz de correr completamente offline en un dispositivo Android de gama media/baja (≥ 4 GB RAM, Snapdragon serie 6xx o equivalente). El agricultor sube una foto de la hoja y la aplicación responde con la clase más probable, su confianza y consejos de manejo. El modelo objetivo no supera los 20 MB y mantiene una latencia ≤ 300 ms por imagen, con cuantización INT8 y un runtime móvil como TensorFlow Lite u ONNX Runtime Mobile.

Esta primera etapa cubre la recolección y documentación del dataset, el análisis exploratorio, el pipeline de preprocesamiento y balanceo, y la comparación de tres arquitecturas candidatas como *baseline* sobre las 9 clases, antes de comprometer el entrenamiento final y el despliegue móvil.

2 Trabajos Previos

Los primeros sistemas de diagnóstico foliar se apoyaban en aprendizaje clásico — SVM, K-Nearest Neighbors, Random Forest — sobre características diseñadas a mano (color, textura, forma de la lesión), con precisiones del 79% al 90% (Autores varios, 2026). La llegada de las CNN cambió la lógica del problema: la red aprende sola la jerarquía de rasgos a partir de los píxeles. Con ese salto se consolidó el F1-score como métrica principal, porque los datasets agrícolas rara vez tienen clases

equilibradas.

El hallazgo más crítico de la literatura es la **brecha de dominio laboratorio/campo**. La mayoría de modelos pioneros se entrenaron con *PlantVillage*, hojas aisladas sobre fondos homogéneos, donde las CNN alcanzan exactitudes de 95.81% a 99.53% (Ahmad et al., 2023a). Pero al evaluarlos contra conjuntos de campo real (*PlantDoc*, *FieldPlant*, *CD&S*) la precisión se desploma al rango de 33.27% a 39.87%. La causa es estructural: en campo, la hoja enferma convive con hojas sanas, malezas, suelo, sombras y reflejos. Con aumento de datos agresivo, mezcla de laboratorio y campo, y eliminación de fondo, arquitecturas como DenseNet169 estabilizan la generalización en 77.50% a 81.60% (Ahmad et al., 2023b). Como se verá en la Sección 5, nuestro EDA confirma el sesgo: `common_rust` tiene el 95.4% de sus imágenes en laboratorio.

Las investigaciones recientes atacan la brecha con esquemas híbridos y en cascada. Los híbridos CNN-ViT combinan la extracción de texturas de las CNN con la capacidad de los Transformers para relacionar lesiones distantes; Shandilya et al. (2025) reportan F1 cercano al 99.13% sobre conjuntos mixtos, aunque sistemas de campo como ZeaWatch ven caer su confianza al 70.8% en condiciones reales (Baya et al., 2024). Otros separan el problema en dos etapas — un detector (Faster R-CNN, LS-RCNN) que aísla la hoja, seguido del clasificador (Liu et al., 2022) — o usan cámaras RGB-D para recortar el fondo antes de clasificar con modelos ligeros como MobileNetV2 (Nan et al., 2023).

Este proyecto aborda la etapa de clasificación, diseñada como componente modular que en el futuro pueda recibir hojas ya segmentadas. El *trade-off* precisión/despliegue justifica elegir arquitecturas ligeras (≤ 20 MB) capaces de inferencia offline, priorizando la evaluación sobre imágenes de campo real.

3 Marco Teórico

3.1 Redes Neuronales Convolucionales

Una CNN procesa datos con estructura de cuadrícula aplicando filtros aprendibles W sobre la imagen I . La convolución discreta en (i, j) es:

$$S(i, j) = (I * W)(i, j) = \sum_m \sum_n I(i - m, j - n) W(m, n) \quad (1)$$

La red aprende una jerarquía visual: las primeras capas responden a bordes y colores, las intermedias a texturas y manchas, las profundas a lesiones con forma y color propios. Dos propiedades la hacen eficiente. La **conectividad local** hace que cada filtro observe una región pequeña, y la **compartición de pesos** reutiliza el mismo filtro por toda la imagen, lo que reduce parámetros y detecta un patrón sin importar dónde aparezca. Las activaciones pasan por no linealidades como ReLU ($f(x) = \max(0, x)$) y por *pooling*, que reduce la resolución espacial e introduce cierta tolerancia a pequeños desplazamientos.

Al final, los mapas de características se proyectan hacia las clases mediante capas lineales. La función *Softmax* convierte los logits z_i en probabilidades:

$$\hat{p}_i = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \quad (2)$$

El entrenamiento minimiza la **Entropía Cruzada**, que castiga la divergencia entre la predicción $\hat{p}$ y la etiqueta real y :

$$\mathcal{L}_{CE} = - \sum_{c=1}^C y_c \log(\hat{p}_c) \quad (3)$$

3.2 Evolución hacia arquitecturas ligeras

La familia de arquitecturas CNN evolucionó desde LeNet y AlexNet hacia diseños más profundos como VGG y ResNet, esta última con conexiones residuales que facilitan entrenar redes muy profundas. Para el despliegue móvil interesa otra rama, la de las redes eficientes. Su pieza clave son las **convoluciones depthwise-separable**, que factorizan una convolución $k \times k$ en un filtro espacial por canal (*depthwise*) y una convolución 1×1 que combina canales (*pointwise*). El ahorro de FLOPs respecto a la convolución estándar es:

$$\frac{\text{Costo separable}}{\text{Costo estándar}} \approx \frac{1}{C_{out}} + \frac{1}{k^2} \quad (4)$$

Para filtros de 3×3 esto significa ser unas 8–9 veces más barato. Sobre esa idea nacen MobileNet, ShuffleNet y EfficientNet, que son la base de las arquitecturas evaluadas en este trabajo.

3.3 Aprendizaje por Transferencia

Cuando no se dispone de millones de imágenes, conviene no partir de cero. El **transfer learning** carga pesos preentrenados en ImageNet (Deng et al., 2009) y los adapta al nuevo dominio: la capa clasificadora original se reemplaza por una ajustada a las clases del problema y los pesos se refinan con descenso de gradiente. La idea no es copiar la tarea original, sino reaprovechar filtros generales de bordes, colores y texturas. Es exactamente lo que hace el pipeline: los tres modelos parten de pesos de ImageNet, se les cambia la capa final por una lineal de 9 salidas y se afina todo el backbone (*fine-tuning* completo).

3.4 Baselines

Un *baseline* es un modelo de referencia sencillo que fija un piso de comparación. Su papel no es ganar, sino marcar el nivel mínimo que las soluciones más complejas deberán superar, y permitir observar a bajo costo el comportamiento de cada arquitectura candidata antes de comprometer un entrenamiento completo. En este proyecto los baselines cumplen ese doble papel: fijan el piso de macro-F1 y sirven de sonda para detectar colapso de clases o sobreajuste temprano sobre un subconjunto reducido, antes de lanzar el pipeline principal.

3.5 Desbalance de clases

Cuando unas clases tienen muchas más imágenes que otras, la red puede tomar el atajo de predecir casi siempre las mayoritarias. Hay tres herramientas habituales, combinables entre sí. La **entropía cruzada ponderada** asigna a cada clase un peso inverso a su frecuencia, de modo que fallar en

una clase rara pese más en el gradiente. El **muestreo ponderado** interviene antes, al armar cada lote, eligiendo con más probabilidad los ejemplos de clases poco frecuentes sin duplicar imágenes en memoria. Y la **pérdida focal** reescala la entropía cruzada para concentrar el aprendizaje en los ejemplos difíciles.

De estas, el proyecto usa el muestreo ponderado en las dos etapas. La diferencia está en la pérdida: los baselines la dejan **sin ponderar** para mantener las corridas simples y comparables entre arquitecturas, mientras que el pipeline principal sí planea combinar el sampler con una **entropía cruzada ponderada** sobre el modelo ya elegido. La mecánica concreta se detalla en la Sección 6.2.

3.6 Métricas de evaluación

En datasets asimétricos, el *accuracy* engaña: un modelo puede parecer bueno acertando solo las clases grandes. Por eso conviene mirar precisión, recall y F1 por clase:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad \text{Recall}_c = \frac{TP_c}{TP_c + FN_c}, \quad F1_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c} \quad (5)$$

El **F1-macro** promedia el F1 de todas las clases sin ponderar por soporte:

$$F1_{\text{macro}} = \frac{1}{C} \sum_{c=1}^C F1_c \quad (6)$$

A diferencia del F1-weighted, el macro exige reconocer la clase con 40 ejemplos con la misma destreza que la de 1 500. El umbral de éxito del proyecto es $F1_{\text{macro}} \geq 0.85$.

3.7 Interpretabilidad

Una CNN es por defecto una caja negra: entrega una etiqueta, no una razón. En este dominio eso importa, porque una clasificación equivocada se traduce en una decisión de manejo equivocada, como fumigar sin necesidad. Necesitamos poder revisar que el modelo mira donde está el problema,

no un atajo del fondo. Las tres técnicas más usadas son LIME, SHAP y Grad-CAM.

LIME (Ribeiro et al., 2016) explica una predicción individual aproximando localmente el modelo con un modelo lineal simple. Para imágenes trabaja con superpíxeles: segmenta la imagen, apaga algunos superpíxeles para generar variantes perturbadas, consulta al modelo por cada una, y ajusta un modelo lineal cuyos coeficientes son la importancia de cada región. Es agnóstico al modelo y muy intuitivo, pero también el más costoso y el menos estable, porque las perturbaciones son aleatorias.

SHAP (Lundberg & Lee, 2017) reparte el crédito de la predicción entre superpíxeles usando los valores de Shapley de la teoría de juegos. Su atractivo es que cumple a la vez varias propiedades deseables, como consistencia, y que sus atribuciones son aditivas: se pueden comparar entre imágenes y agregar para una lectura global. A cambio, sus variantes precisas son caras o dependen de una línea base cuidadosa.

Grad-CAM (Selvaraju et al., 2017) es específico de CNN. Toma la última capa convolucional, retropropaga el gradiente del logit objetivo hasta sus mapas de activación y produce un mapa de calor de baja resolución que señala qué regiones activaron la clase. Es órdenes de magnitud más rápido, determinista y muy legible, aunque localiza regiones amplias en vez de bordes finos.

Tabla 1: Comparación de las tres técnicas de interpretabilidad

Criterio	LIME	SHAP	Grad-CAM
Tipo	Agnóstico	Agnóstico o específico	Específico de CNN
Alcance	Local	Local, agregable a global	Local
Costo por imagen	Alto	Medio	Muy bajo
Estabilidad	Baja	Alta	Alta
Fundamento	Heurístico	Axiomático (Shapley)	Basado en gradientes

Sobre esta base, el proyecto adopta la pareja **LIME + Grad-CAM** en los baselines: LIME aporta la lectura por superpíxeles con signo y Grad-CAM un mapa de calor rápido y determinista, suficientes para auditar caso por caso a bajo costo. **SHAP** se reserva para el pipeline principal,

donde su fundamento más fuerte y su capacidad de agregarse a una vista global valen la pena una vez que hay un modelo final estable. En ambas etapas el análisis es post-hoc: se ejecuta bajo demanda sobre un modelo ya entrenado, no se acopla al ciclo de entrenamiento.

4 Descripción del Dataset

El dataset consolidado en `clean/` reúne **31 622 imágenes** en **9 clases**, integradas y depuradas a partir de 6 fuentes públicas (Tabla 2). Cada imagen queda etiquetada por entorno de captura: `lab` (fondo controlado) o `real` (campo abierto).

Tabla 2: Fuentes de datos y su licencia

Fuente	Dominio	Licencia
Maize in Field Dataset	Campo real	CC BY-NC-SA 4.0
Maize Dis-eases	Lab + campo	CC BY-NC-SA 4.0
CropDG Unified Multi-Domain	Multi-dominio	CC BY-NC-SA 4.0
Maize / Beans / Tomatoes – África	Campo real	Apache 2.0 + CC
Maize Nutrient Deficiency	Campo real	CC BY 4.0
Corn Leaf – Roboflow	Campo real	CC BY 4.0

Dos fuentes más fueron descartadas: *Corn Leaf Diseases* (MIT), que solo distribuye imágenes aumentadas sintéticamente sin conservar los originales; y *Multicrop Disease*, por mezclar sin metadatos imágenes preprocesadas, de laboratorio y de campo, imposibles de separar con confianza.

Durante la limpieza se detectaron duplicados *entre* fuentes mediante hash perceptual (pHash, `imagededup`), eliminando **8 538 imágenes** en **8 050 grupos**, sobre todo entre *maize_disease* ($\sim 6\,508$) y *multi_disease* ($\sim 1\,980$). Sin esa depuración, copias idénticas habrían aparecido en train y validación, inflando las métricas por fuga de datos. La Tabla 3 muestra la composición final.

Tabla 3: Composición del dataset limpio por clase

Clase	Lab	Real	Total
healthy	0	8 744	8 744
northern_corn_leaf_blight	888	5 942	6 830
lethal_necrosis	0	6 415	6 415
fall_armyworm	0	4 857	4 857
common_rust	2 150	106	2 256
gray_leaf_spot	513	606	1 119
phosphorus_deficiency	0	612	612
nitrogen_deficiency	0	523	523
potassium_deficiency	0	266	266
Total	3 551	28 071	31 622

5 Análisis Exploratorio de Datos

El EDA completo, reproducible en `notebooks/01_eda.ipynb`, busca responder tres preguntas: qué hay en el dataset, qué problemas tiene y qué decisiones impone. De ahí salieron cinco riesgos principales.

Desbalance de clases. La clase `healthy` (8 744 imágenes) supera en **32.9×** a `potassium_deficiency` (266); ver Figura 1. Sin intervención, un modelo tendería a ignorar las minoritarias, que son justo las de diagnóstico más delicado.

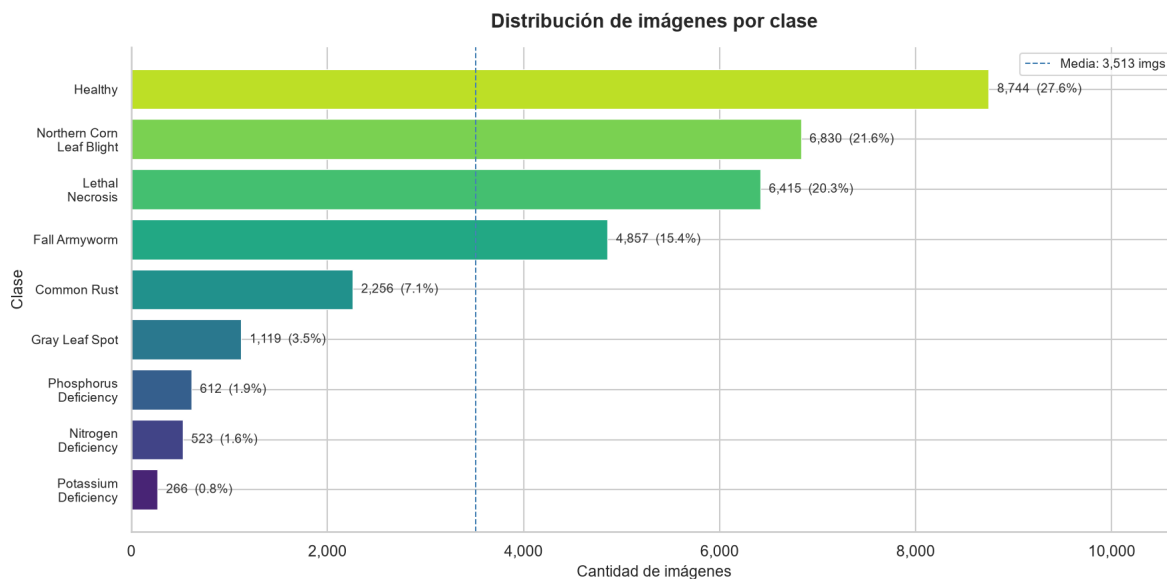


Figura 1: Distribución de imágenes por clase en `clean/`.

Sesgo de dominio laboratorio/campo. La proporción lab/real varía mucho entre clases (Figura 2); el caso crítico es `common_rust`, con **95.4%** de imágenes en entorno controlado. Un modelo descuidado podría aprender el fondo de laboratorio en lugar del síntoma. Esta señal aparece también en el brillo: la mediana de `common_rust` (~ 95) queda muy por debajo del resto (~ 120 – 140), reflejo directo de sus fondos oscuros de laboratorio.

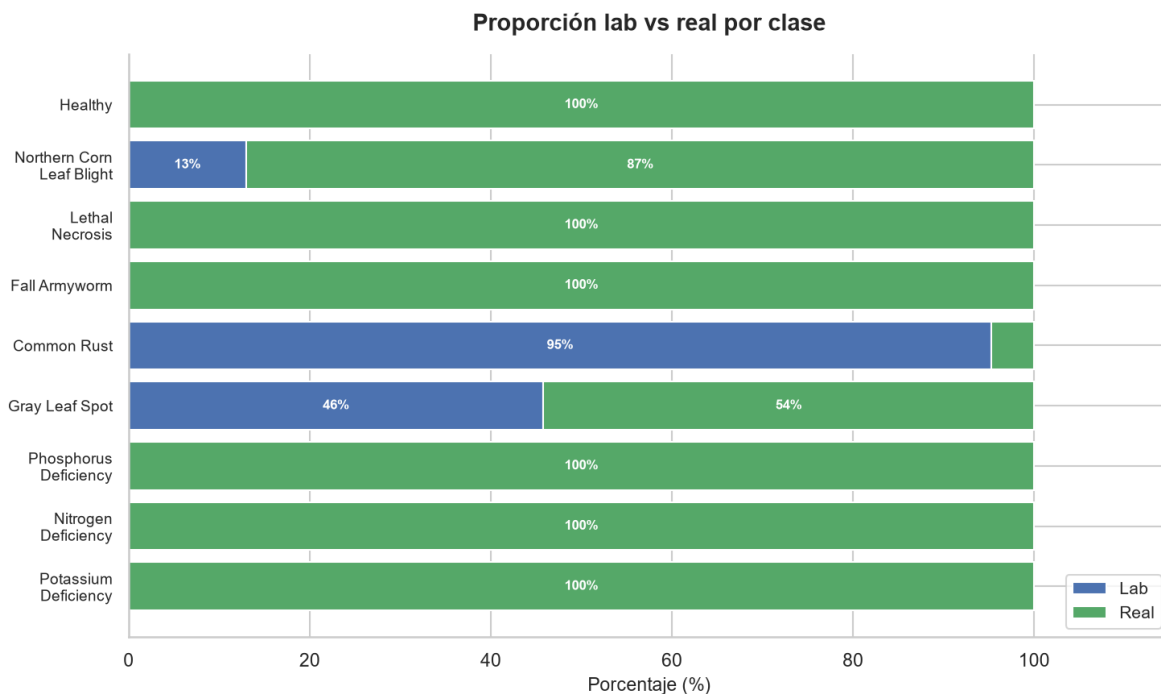


Figura 2: Proporción de imágenes lab vs. real por clase.

Fuente única en clases pequeñas. Las tres deficiencias nutricionales provienen de una sola fuente (`maize_nutrient`) y `lethal_necrosis` depende por completo de `maize_africa`. Esa dependencia limita la diversidad de condiciones de captura y suma riesgo de sobreajuste.

Resolución y calidad. Las resoluciones son marcadamente multimodales: un pico en ~ 256 px (laboratorio de PlantVillage), otro en ~ 640 px (Roboflow y deficiencias) y una cola larga hacia $3000\text{--}5000+$ px (campo). Todo se lleva a 224×224 px como primera transformación, con lo que las imágenes grandes pierden detalle fino de las lesiones. En calidad, el único problema relevante es el desenfoque, y molesta que golpee sobre todo a clases minoritarias: `gray_leaf_spot` ($\sim 38\%$), `nitrogen_deficiency` ($\sim 28\%$) y `potassium_deficiency` ($\sim 21\%$). La sub y sobreexposición son prácticamente inexistentes.

Sesgos combinados. La Figura 3 resume el cuadro. Un caso aparte es `fall_armyworm`, cuyo sesgo es cualitativo: mezcla dos señales según la fuente — hoja con daño sin insecto visible, y hoja con daño y gusano visible. Se mantuvieron mezcladas a propósito, porque la clase y su recomendación dependen del patrón de daño, no del insecto; a cambio, hay que vigilar de cerca la

validación para que el modelo no aprenda el gusano como atajo.

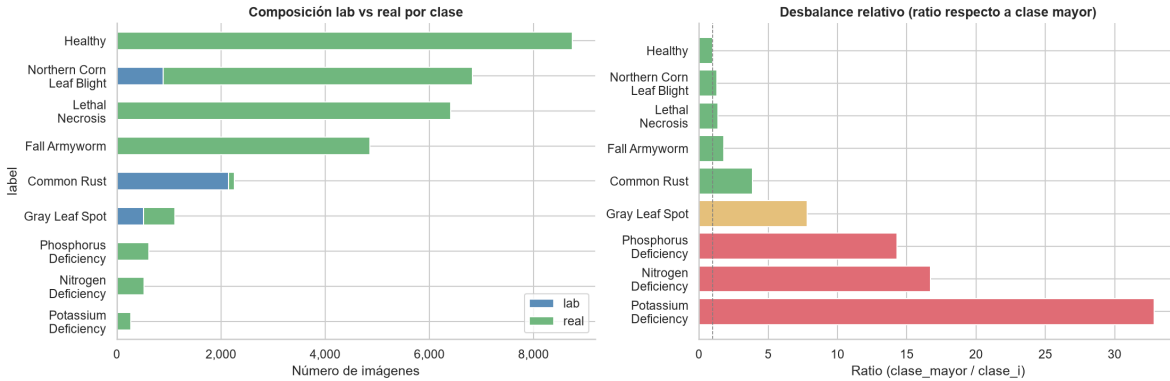


Figura 3: Composición lab/real y desbalance relativo por clase.

De aquí salen dos decisiones firmes: usar muestreo ponderado con augmentation dirigido a las minoritarias, y medir la generalización sobre imágenes de campo real, no de laboratorio.

6 Preprocesamiento, Feature Engineering y Estrategia

Al ser clasificación de imágenes con transfer learning, no hay ingeniería de *features* manual: las representaciones las aprende el backbone preentrenado en ImageNet. El esfuerzo se concentró en normalización, división de datos, balanceo y en definir la estrategia de entrenamiento de cada etapa.

6.1 Normalización y división

Todo tensor pasa por un único punto de entrada (`load_and_normalize_image()`): corrección EXIF de orientación, conversión a RGB estricta y normalización con media/desviación de ImageNet ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$), requerida por los backbones. La división es estratificada 70/15/15 (train/val/test) por `label + environment`, con semilla fija (42), para que la mezcla lab/real sea consistente en los tres conjuntos.

6.2 Balanceo

El balanceo opera en tres capas. Primero, un tope configurable por clase (1 500 en el perfil baseline) que recorta solo las mayoritarias y deja intactas las minoritarias. Segundo, `WeightedRandomSampler`, que asigna a cada muestra un peso $1/n_c$ e iguala la frecuencia efectiva por época sin duplicar archivos. Tercero, augmentation extendido (`CornMinorityTransforms`) reservado a clases con ratio de desbalance $> 4\times$: recortes, rotación de hasta $\pm 30^\circ$, `ColorJitter` más agresivo y `GaussianBlur` suave, manteniendo el matiz casi intacto porque el diagnóstico de deficiencias depende del color. La pérdida se deja deliberadamente **sin ponderar**: ponderar sampler, augmentation y pérdida a la vez sobre-compensaría el mismo desbalance por tres vías. Validación y prueba usan solo `Resize` + normalización, sin aleatoriedad.

6.3 Baselines elegidos y por qué

Se seleccionaron tres redes ligeras preentrenadas en ImageNet, cada una en un punto distinto del eje precisión–eficiencia, y las tres convertibles a TFLite para el despliegue móvil.

EfficientNet-B0 (Tan & Le, 2019) introduce el *compound scaling*, que escala profundidad, ancho y resolución a la vez con un coeficiente ϕ . Usa bloques MBConv con módulos Squeeze-and-Excitation y alcanza $\sim 77.1\%$ Top-1 con 5.3 M de parámetros. Representa el extremo de mayor precisión.

EfficientNet-Lite0 (Google Brain, 2020) es una variante de B0 para hardware perimetral. Elimina los bloques SE, reemplaza Swish por ReLU6 (que estabiliza la cuantización INT8, evitando la caída de $\sim 75\%$ a $\sim 46\%$ que produce Swish sin sustituir) y evita operaciones incompatibles con ciertos compiladores. Alcanza $\sim 74.9\%$ Top-1 con 4.7 M de parámetros. Representa el extremo de máxima facilidad de cuantización.

ShuffleNetV2-x1.0 (Ma et al., 2018) parte de que los FLOPs no equivalen a velocidad por el costo de acceso a memoria. Su *channel split + channel shuffle* reduce ese costo al mínimo, y la hace la más veloz y liviana del grupo ($\sim 69.4\%$ Top-1, 2.3 M parámetros, ~ 5 MB serializado).

En los tres casos se reemplaza solo la capa clasificadora final por una lineal de 9 salidas y se deja el resto del backbone entrenable (*fine-tuning* completo). La Tabla 4 resume el *trade-off*.

Tabla 4: Arquitecturas baseline comparadas

Modelo	Parám.	Top-1 IN	Tamaño	Perfil
EfficientNet-B0	5.3 M	~77.1%	~16 MB	Máxima precisión
EfficientNet-Lite0	4.7 M	~74.9%	~14 MB	Cuantización INT8
ShuffleNetV2-x1.0	2.3 M	~69.4%	~5 MB	Máxima eficiencia

6.4 Estrategia de entrenamiento: baselines vs. principal

La infraestructura de datos y de modelos es común a las dos etapas; lo que cambia es cuánto se afina el loop de entrenamiento. Los baselines usan una configuración deliberadamente sencilla para que las diferencias de resultado se deban a la arquitectura y no a trucos de entrenamiento: learning rate fijo, un número de épocas cerrado y pérdida sin ponderar. El pipeline principal, ya sobre la arquitectura elegida, incorpora los mecanismos de optimización y regularización que la teoría respalda (Tabla 5).

Tabla 5: Configuración de entrenamiento por etapa

Elemento	Baselines (esta etapa)	Pipeline principal (previsto)
Datos	9 clases, cap 1 500/clase	9 clases, 100% de imágenes
Learning rate	Fijo (10^{-4})	Scheduler (cosine o plateau) con warmup
Parada	30 épocas fijas	Early stopping (<i>patience</i> 5–8), <i>best.pth</i>
Pérdida	CrossEntropy sin ponderar	CrossEntropy ponderada + <i>label smoothing</i>
Balanceo	Sampler + augmentation minoritaria	Sampler + pérdida ponderada
Optimizador	AdamW	AdamW + <i>gradient clipping</i>
Interpretabilidad	LIME + Grad-CAM	+ SHAP (vista global)

El *scheduler* con warmup ataca dos cosas vistas antes: un lr fijo mantiene pasos demasiado grandes en las épocas finales, cuando el modelo debería estar afinando, y el warmup da margen a

arquitecturas que arrancan lento. El *early stopping* recorta cómputo sin sacrificar calidad, porque se conserva el mejor checkpoint. Y el *label smoothing* evita que el modelo se vuelva sobreconfiado, algo directamente relevante para el caso de uso, ya que en los baselines se observa justamente que falla con alta confianza. La métrica primaria sigue siendo el **macro-F1** con umbral 0.85 en ambas etapas.

7 Pipeline de Baselines y Resultados

7.1 Dataset y configuración

Los baselines se entrenan sobre el perfil `baseline`: las 9 clases con tope de 1 500 imágenes por clase, aplicado *antes* de dividir. Solo se recortan las mayoritarias; las minoritarias quedan intactas (potasio 266, nitrógeno 523, fósforo 612). El resultado es un subconjunto de 10 020 imágenes (Tabla 6) que conserva el desbalance natural. El test es mayoritariamente de campo real (1 182 imágenes reales vs. 321 de laboratorio), lo que mide el desempeño en condiciones agrícolas.

Tabla 6: Composición del dataset baseline (9 clases, cap 1 500/clase)

Split	Imágenes
Entrenamiento (70%)	7 014
Validación (15%)	1 503
Prueba (15%)	1 503
Total	10 020

Los tres modelos comparten la configuración baseline de la Tabla 5 (30 épocas, batch 32, AdamW con $\text{lr } 10^{-4}$, pérdida sin ponderar), de modo que las diferencias vengan de la arquitectura y no del entrenamiento. Cada corrida se versiona en `outputs/baselines/<modelo>/<run_id>/`. Con el cap de 1 500, solo `potassium_deficiency` cruza el umbral de $4\times$ que activa la *augmentation* extendida.

7.2 Resultados globales

La Tabla 7 resume el desempeño en prueba (1 503 imágenes, 9 clases). Los tres superan la meta con holgura y quedan a apenas 2 puntos entre sí, así que en calidad son esencialmente equivalentes. EfficientNet-B0 lidera el macro-F1; ShuffleNetV2-x1.0 obtiene la menor loss y el checkpoint más liviano, $\sim 3\times$ menor que B0.

Tabla 7: Comparación de modelos baseline en el conjunto de prueba

Modelo	Accuracy	macro-F1	Loss (test)	Checkpoint
EfficientNet-B0	0.9521	0.9146	0.229	15.6 MB
ShuffleNetV2-x1.0	0.9508	0.9030	0.169	5.0 MB
EfficientNet-Lite0	0.9474	0.8951	0.218	13.1 MB

7.3 Dónde falla el modelo

La cifra agregada esconde que el rendimiento no es parejo. Casi todo el error se concentra en una clase: `potassium_deficiency` (F1 0.49–0.62 con solo 40 imágenes en test), que por sí sola arrastra el macro-F1 entre 3.7 y 5.0 puntos; excluida, los tres modelos operan cerca de 0.95. El resto del error es *dirigido*: las tres deficiencias N/P/K se confunden entre sí pero el 97% de sus predicciones se queda dentro de ese propio bloque, porque comparten el síntoma base de clorosis. Hay además una confusión menor pero consistente entre `gray_leaf_spot` y `northern_corn_leaf_blight`, de lesiones alargadas parecidas. Fuera de eso, `common_rust`, `lethal_necrosis` y `healthy` van con recall ≥ 0.99 .

La lectura práctica es clara: el baseline no necesita más capacidad de modelo, sino resolver un cuello de botella de datos muy localizado en potasio.

7.4 Interpretabilidad aplicada

Aplicando la pareja LIME + Grad-CAM definida antes, en las clases fáciles la atribución se concentra sobre el tejido de la hoja (Figura 4), indicio de que el modelo no se apoya en atajos

del fondo. El panel más revelador es un error: potasio clasificado como nitrógeno con 99.7% de confianza (Figura 5), donde Grad-CAM se enciende sobre una punta necrótica real, pero ambigua entre las tres deficiencias. El modelo mira el lugar correcto y aun así se equivoca.

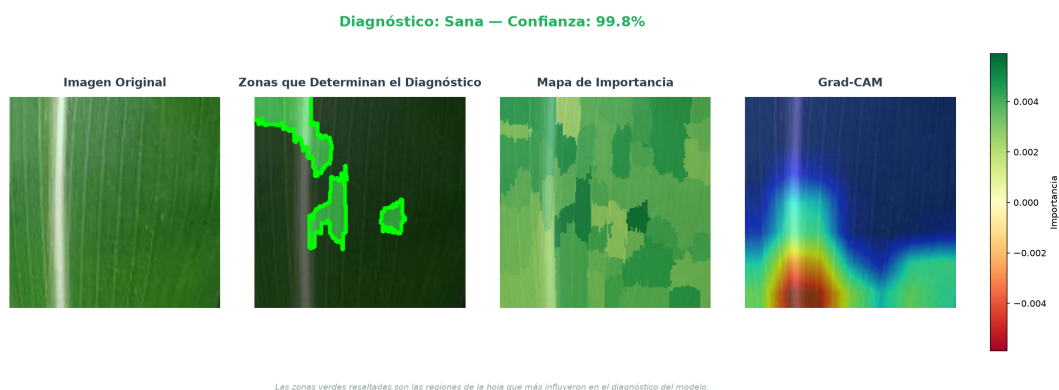


Figura 4: Explicación LIME de un acierto de ShuffleNetV2-x1.0 (healthy, campo real).



Figura 5: LIME + Grad-CAM: potasio clasificado como nitrógeno con alta confianza (EfficientNet-B0).

La confianza agregada lo confirma: en EfficientNet-B0, los aciertos promedian 0.987 y los fallos 0.914, con los errores de potasio aún por encima de 0.89. Dicho de otro modo, **el modelo se equivoca sin dudar**, así que un aviso del tipo “confianza baja, consulte a un técnico” no capturaría estos casos. Una alternativa más prometedora sería agrupar N/P/K en una respuesta única de “deficiencia nutricional” cuando la predicción caiga dentro de ese bloque.

Nota: la figura del acierto healthy proviene de una corrida preliminar de 4 clases y es

solo cualitativa. El análisis sistemático sobre las 9 clases, con fidelidad agregada y auditoría de estabilidad de LIME, queda para el trabajo futuro.

7.5 Reproducibilidad

Todo el pipeline es reproducible desde el repositorio del proyecto. El código se instala como paquete editable y la configuración de dominio (clases, `target_size`, semilla, perfil baseline) vive centralizada en un único archivo, sin constantes dispersas. La semilla está fijada (42) en la generación de splits, el muestreo y el entrenamiento, así que los cortes train/val/test y las métricas se regeneran igual entre máquinas. Cada corrida se versiona con sus pesos, historial, predicciones por imagen y reportes. Las etapas se disparan con comandos únicos — generar splits, entrenar los baselines, y producir los reportes de interpretabilidad —, y el entrenamiento corre tanto en local como en GPU en la nube con el mismo código. La documentación ampliada, con el detalle de cada etapa, está en el sitio del proyecto.

8 Conclusiones y Trabajo Futuro

Sobre las 9 clases con tope de 1 500 imágenes, los tres baselines superan la meta de macro-F1 ≥ 0.85 y son esencialmente equivalentes en calidad, lo que valida tanto las arquitecturas candidatas como el pipeline de datos que las alimenta.

El hallazgo central es que `potassium_deficiency` es el único cuello de botella que impide subir la métrica: excluida esa clase, los tres modelos operan cerca de 0.95. El modelo ya detecta la categoría “deficiencia nutricional” con alta fiabilidad, pero no separa N/P/K por la escasez de datos de potasio, y encima falla con alta confianza. Es un problema de datos, no de capacidad. Para la Etapa 2, las prioridades son:

- **Más datos de potasio.** Recolectar imágenes reales adicionales, augmentation dirigido o pérdida ponderada por clase.

- **Interpretabilidad sobre 9 clases.** Extender LIME y Grad-CAM con fidelidad agregada y auditoría de estabilidad, e incorporar SHAP en el pipeline principal.
- **Arquitectura para despliegue.** Siendo los tres equivalentes en calidad, la decisión debe pesar tamaño y latencia en hardware objetivo, lo que por ahora favorece a ShuffleNetV2-x1.0 (5 MB, menor loss). Falta probar con el modelo exportado a TFLite en dispositivo real.
- **Entrenamiento completo.** Entrenar sobre las 31 622 imágenes sin tope, con el balanceo refinado a partir de estos hallazgos.

9 Referencias

- Ahmad, A., El Gamal, A., & Saraswat, D. (2023a). Toward generalization of deep learning-based plant disease identification under controlled and field conditions. *IEEE Access*, 11, 9042–9057.
- Ahmad, A., El Gamal, A., & Saraswat, D. (2023b). Towards generalization of deep learning-based plant disease identification under controlled and field conditions. Purdue e-Pubs. <https://docs.lib.purdue.edu/fund/58/>
- Autores varios. (2026). Research advances in maize crop disease detection using machine learning and deep learning approaches. *Computers*, 15(2), art. 99. <https://www.mdpi.com/2073-431X/15/2/99>
- Baya, J. et al. (2024). A mobile-based system for maize leaf disease detection and classification using machine learning algorithms and deep learning: A case study of ZeaWatch. Preprint.
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. En *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 248–255). Miami, FL, USA.

- Google Brain / TensorFlow Team. (2020, marzo). *Higher accuracy on vision models with EfficientNet-Lite*. TensorFlow Blog. <https://blog.tensorflow.org/2020/03/higher-accuracy-on-vision-models-with-efficientnet-lite.html>
- Hu, J., Shen, L., & Sun, G. (2018). Squeeze-and-excitation networks. En *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 7132–7141). Salt Lake City, UT, USA.
- Liu, H., Lv, H., Li, J., Liu, Y., & Deng, L. (2022). Research on maize disease identification methods in complex environments based on cascade networks and two-stage transfer learning. *Scientific Reports*, 12(1), art. 18914.
- Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. En *Advances in Neural Information Processing Systems (NeurIPS)*, 30, 4765–4774. Long Beach, CA, USA.
- Ma, N., Zhang, X., Zheng, H.-T., & Sun, J. (2018). ShuffleNet V2: Practical guidelines for efficient CNN architecture design. En *Proceedings of the European Conference on Computer Vision (ECCV)* (pp. 116–131). Munich, Germany.
- Molnar, C. (2022). *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable* (2nd ed.). <https://christophm.github.io/interpretable-machine-learning-book/>
- Nan, F. et al. (2023). A novel method for maize leaf disease classification using the RGB-D post-segmentation image data. *Frontiers in Plant Science*, 14, art. 1268015.
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). “Why should I trust you?”: Explaining the predictions of any classifier. En *Proceedings of the 22nd ACM SIGKDD International*

Conference on Knowledge Discovery and Data Mining (KDD) (pp. 1135–1144). San Francisco, CA, USA.

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual explanations from deep networks via gradient-based localization. En *Proceedings of the IEEE International Conference on Computer Vision (ICCV)* (pp. 618–626). Venice, Italy.

Shandilya et al. (2025). Enhanced maize leaf disease detection and classification using an integrated CNN-ViT model. *Food Science & Nutrition*. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12208913/>

Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. En *Proceedings of the 36th International Conference on Machine Learning (ICML)* (pp. 6105–6114). Long Beach, CA, USA.